



```
In [ ]: import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, Flatten
        import numpy as np

        # Load dataset
        (X_train, y_train), (X_test, y_test) = mnist.load_data()

        # Normalize
        X_train = X_train / 255.0
        X_test = X_test / 255.0

        # Flatten
        X_train = X_train.reshape(-1, 784)
        X_test = X_test.reshape(-1, 784)

        # One-hot encoding
        y_train = to_categorical(y_train, 10)
        y_test = to_categorical(y_test, 10)

        # Model
        model = Sequential([
            Dense(10, activation='softmax', input_shape=(784,))
        ])

        model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

        model.summary()

        model.fit(X_train, y_train, epochs=10, batch_size=64)

        test_loss, test_acc = model.evaluate(X_test, y_test)
        print("Test accuracy:", test_acc)
```

Model: "sequential_12"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

```

Epoch 1/10
938/938 ————— 2s 2ms/step - accuracy: 0.7833 - loss: 0.8455
Epoch 2/10
938/938 ————— 2s 2ms/step - accuracy: 0.9091 - loss: 0.3321
Epoch 3/10
938/938 ————— 2s 2ms/step - accuracy: 0.9178 - loss: 0.2950
Epoch 4/10
938/938 ————— 2s 2ms/step - accuracy: 0.9200 - loss: 0.2885
Epoch 5/10
938/938 ————— 2s 2ms/step - accuracy: 0.9258 - loss: 0.2668
Epoch 6/10
938/938 ————— 2s 2ms/step - accuracy: 0.9252 - loss: 0.2713
Epoch 7/10
938/938 ————— 2s 2ms/step - accuracy: 0.9275 - loss: 0.2614
Epoch 8/10
938/938 ————— 2s 2ms/step - accuracy: 0.9280 - loss: 0.2583
Epoch 9/10
938/938 ————— 2s 2ms/step - accuracy: 0.9292 - loss: 0.2540
Epoch 10/10
938/938 ————— 2s 2ms/step - accuracy: 0.9301 - loss: 0.2536
313/313 ————— 1s 2ms/step - accuracy: 0.9177 - loss: 0.2988
Test accuracy: 0.9279999732971191

```

```

In [ ]: import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, Flatten
        import numpy as np

        # Load dataset
        (X_train, y_train), (X_test, y_test) = mnist.load_data()

        # Normalize
        X_train = X_train / 255.0
        X_test = X_test / 255.0

        # Flatten
        X_train = X_train.reshape(-1, 784)
        X_test = X_test.reshape(-1, 784)

        # One-hot encoding
        y_train = to_categorical(y_train, 10)
        y_test = to_categorical(y_test, 10)

        # Model
        model = Sequential([
            Dense(10, activation='softmax', input_shape=(784,))
        ])

        model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

```

```
model.summary()

model.fit(X_train, y_train, epochs=20, batch_size=64)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_13"

Layer (type)	Output Shape	Param #
dense_13 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

```

Epoch 1/20
938/938 ————— 2s 2ms/step - accuracy: 0.7732 - loss: 0.8701
Epoch 2/20
938/938 ————— 2s 2ms/step - accuracy: 0.9067 - loss: 0.3337
Epoch 3/20
938/938 ————— 2s 2ms/step - accuracy: 0.9147 - loss: 0.3069
Epoch 4/20
938/938 ————— 2s 2ms/step - accuracy: 0.9205 - loss: 0.2872
Epoch 5/20
938/938 ————— 2s 2ms/step - accuracy: 0.9228 - loss: 0.2757
Epoch 6/20
938/938 ————— 2s 2ms/step - accuracy: 0.9246 - loss: 0.2672
Epoch 7/20
938/938 ————— 2s 2ms/step - accuracy: 0.9267 - loss: 0.2633
Epoch 8/20
938/938 ————— 2s 2ms/step - accuracy: 0.9264 - loss: 0.2620
Epoch 9/20
938/938 ————— 2s 2ms/step - accuracy: 0.9286 - loss: 0.2558
Epoch 10/20
938/938 ————— 2s 2ms/step - accuracy: 0.9295 - loss: 0.2568
Epoch 11/20
938/938 ————— 2s 2ms/step - accuracy: 0.9296 - loss: 0.2504
Epoch 12/20
938/938 ————— 2s 2ms/step - accuracy: 0.9302 - loss: 0.2486
Epoch 13/20
938/938 ————— 2s 2ms/step - accuracy: 0.9314 - loss: 0.2438
Epoch 14/20
938/938 ————— 2s 2ms/step - accuracy: 0.9313 - loss: 0.2485
Epoch 15/20
938/938 ————— 2s 2ms/step - accuracy: 0.9326 - loss: 0.2477
Epoch 16/20
938/938 ————— 2s 2ms/step - accuracy: 0.9322 - loss: 0.2423
Epoch 17/20
938/938 ————— 2s 2ms/step - accuracy: 0.9327 - loss: 0.2442
Epoch 18/20
938/938 ————— 2s 2ms/step - accuracy: 0.9329 - loss: 0.2426
Epoch 19/20
938/938 ————— 2s 2ms/step - accuracy: 0.9334 - loss: 0.2364
Epoch 20/20
938/938 ————— 2s 2ms/step - accuracy: 0.9345 - loss: 0.2404
313/313 ————— 1s 1ms/step - accuracy: 0.9160 - loss: 0.2986
Test accuracy: 0.926800012588501

```

```

In [ ]: import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense
        import matplotlib.pyplot as plt

        # Load dataset
        (X_train, y_train), (X_test, y_test) = mnist.load_data()

        # Normalize

```

```

X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model with hidden layer
model = Sequential([
    Dense(32, activation='relu', input_shape=(784,)), # Hidden layer
    Dense(10, activation='softmax') # Output layer
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train model
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# Plot Accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'])
plt.show()

# Plot Loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'])
plt.show()

```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 32)	25,120
dense_15 (Dense)	(None, 10)	330

Total params: 25,450 (99.41 KB)

Trainable params: 25,450 (99.41 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

750/750  **4s** 4ms/step - accuracy: 0.7890 - loss: 0.7567 - val_accuracy: 0.9287 - val_loss: 0.2571

Epoch 2/10

750/750  **2s** 2ms/step - accuracy: 0.9292 - loss: 0.2507 - val_accuracy: 0.9438 - val_loss: 0.2068

Epoch 3/10

750/750  **2s** 2ms/step - accuracy: 0.9441 - loss: 0.1999 - val_accuracy: 0.9481 - val_loss: 0.1869

Epoch 4/10

750/750  **2s** 2ms/step - accuracy: 0.9504 - loss: 0.1755 - val_accuracy: 0.9536 - val_loss: 0.1687

Epoch 5/10

750/750  **2s** 2ms/step - accuracy: 0.9566 - loss: 0.1457 - val_accuracy: 0.9542 - val_loss: 0.1572

Epoch 6/10

750/750  **2s** 2ms/step - accuracy: 0.9637 - loss: 0.1271 - val_accuracy: 0.9569 - val_loss: 0.1452

Epoch 7/10

750/750  **2s** 3ms/step - accuracy: 0.9654 - loss: 0.1161 - val_accuracy: 0.9592 - val_loss: 0.1393

Epoch 8/10

750/750  **2s** 3ms/step - accuracy: 0.9670 - loss: 0.1133 - val_accuracy: 0.9603 - val_loss: 0.1355

Epoch 9/10

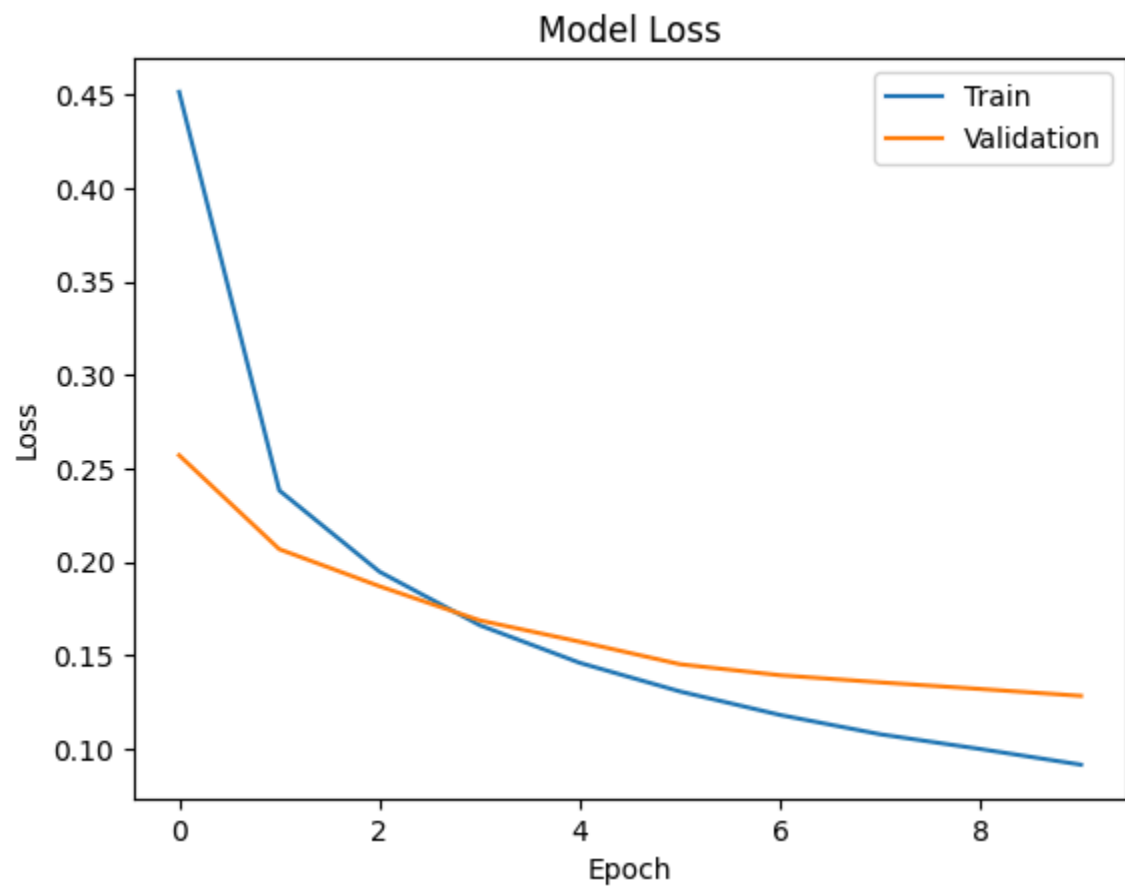
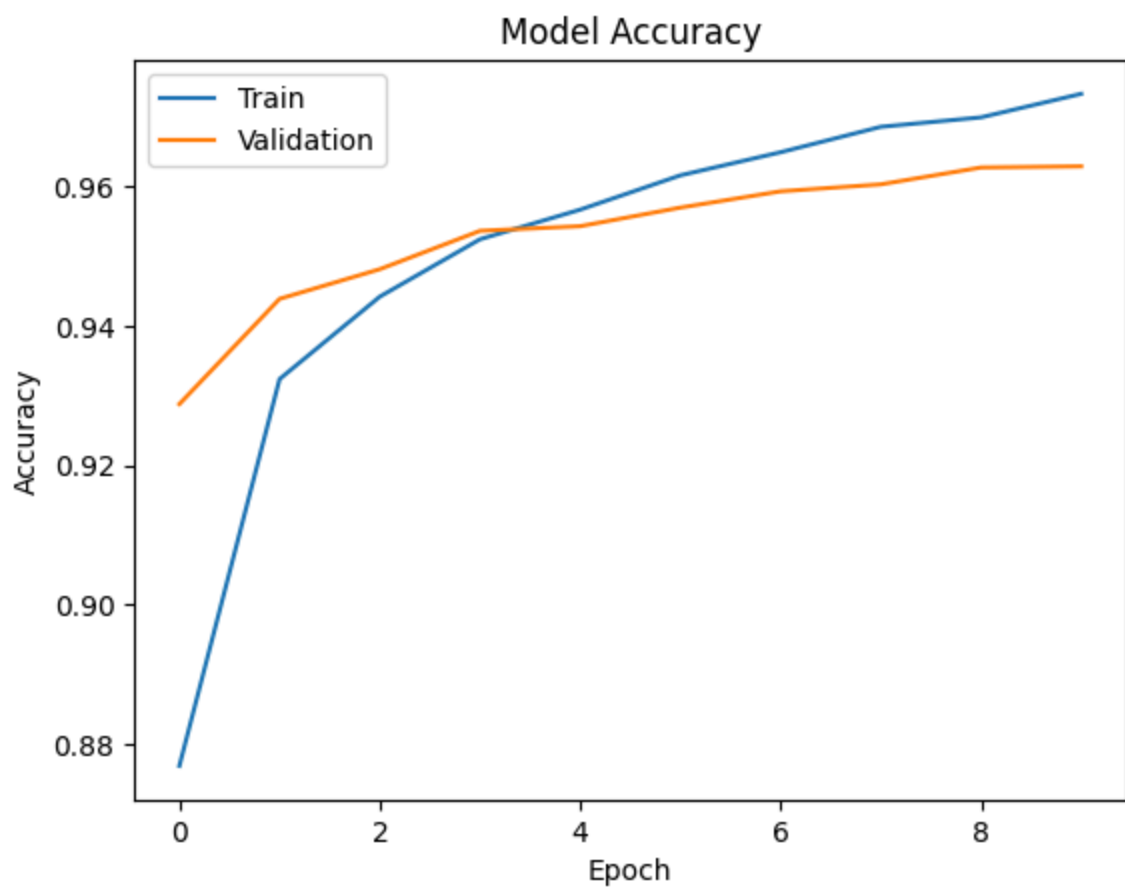
750/750  **2s** 2ms/step - accuracy: 0.9705 - loss: 0.0992 - val_accuracy: 0.9627 - val_loss: 0.1319

Epoch 10/10

750/750  **2s** 2ms/step - accuracy: 0.9730 - loss: 0.0908 - val_accuracy: 0.9628 - val_loss: 0.1283

313/313  **0s** 1ms/step - accuracy: 0.9598 - loss: 0.1352

Test accuracy: 0.9649999737739563



```

In [ ]: import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense
        import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model with two hidden layers
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)), # Hidden Layer 1
    Dense(32, activation='relu'), # Hidden Layer 2
    Dense(10, activation='softmax') # Output Layer
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train model
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# Accuracy Graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph
plt.plot(history.history['loss'])

```



```
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

Model: "sequential_15"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 64)	50,240
dense_17 (Dense)	(None, 32)	2,080
dense_18 (Dense)	(None, 10)	330

Total params: 52,650 (205.66 KB)

Trainable params: 52,650 (205.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

750/750 ————— **3s** 3ms/step - accuracy: 0.8122 - loss: 0.6613 - val_accuracy: 0.9457 - val_loss: 0.1882

Epoch 2/10

750/750 ————— **3s** 4ms/step - accuracy: 0.9505 - loss: 0.1690 - val_accuracy: 0.9612 - val_loss: 0.1392

Epoch 3/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9638 - loss: 0.1168 - val_accuracy: 0.9649 - val_loss: 0.1184

Epoch 4/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9721 - loss: 0.0941 - val_accuracy: 0.9661 - val_loss: 0.1145

Epoch 5/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9763 - loss: 0.0779 - val_accuracy: 0.9681 - val_loss: 0.1063

Epoch 6/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9800 - loss: 0.0632 - val_accuracy: 0.9678 - val_loss: 0.1072

Epoch 7/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9831 - loss: 0.0559 - val_accuracy: 0.9686 - val_loss: 0.1060

Epoch 8/10

750/750 ————— **3s** 4ms/step - accuracy: 0.9854 - loss: 0.0473 - val_accuracy: 0.9695 - val_loss: 0.1103

Epoch 9/10

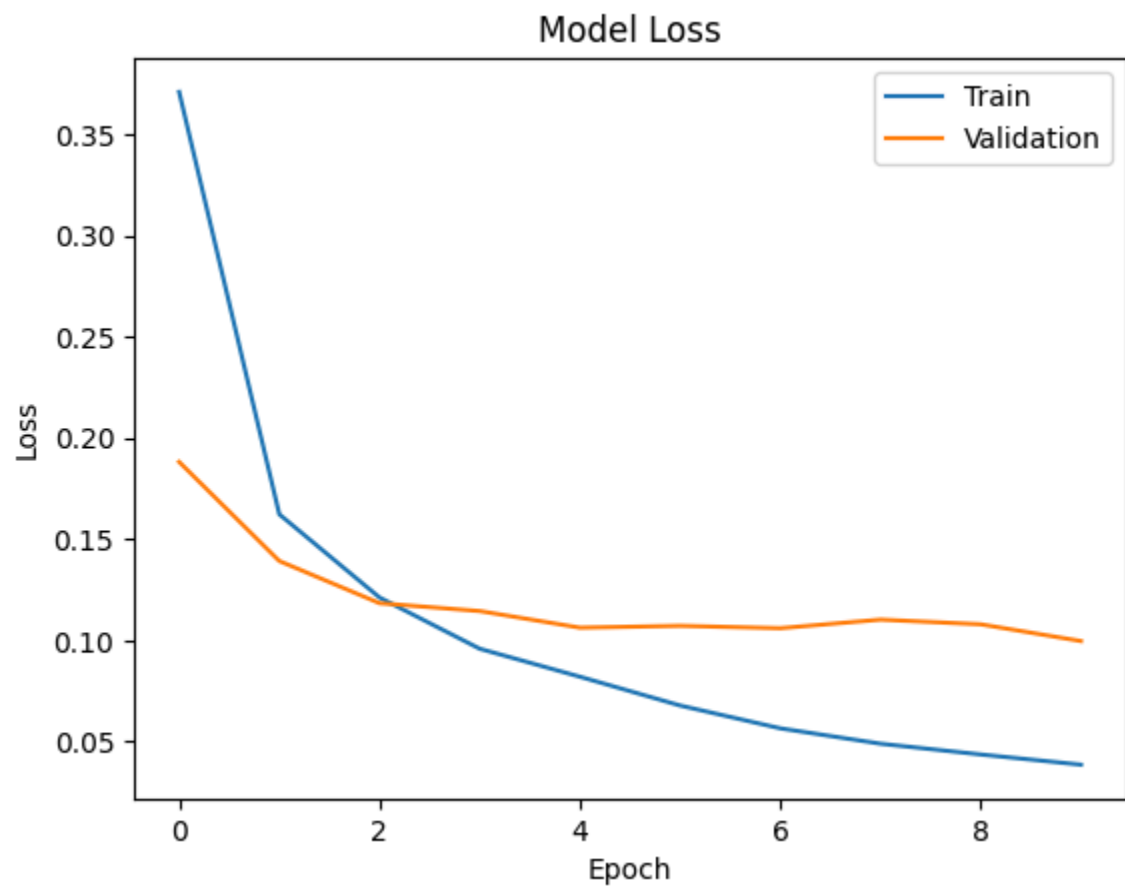
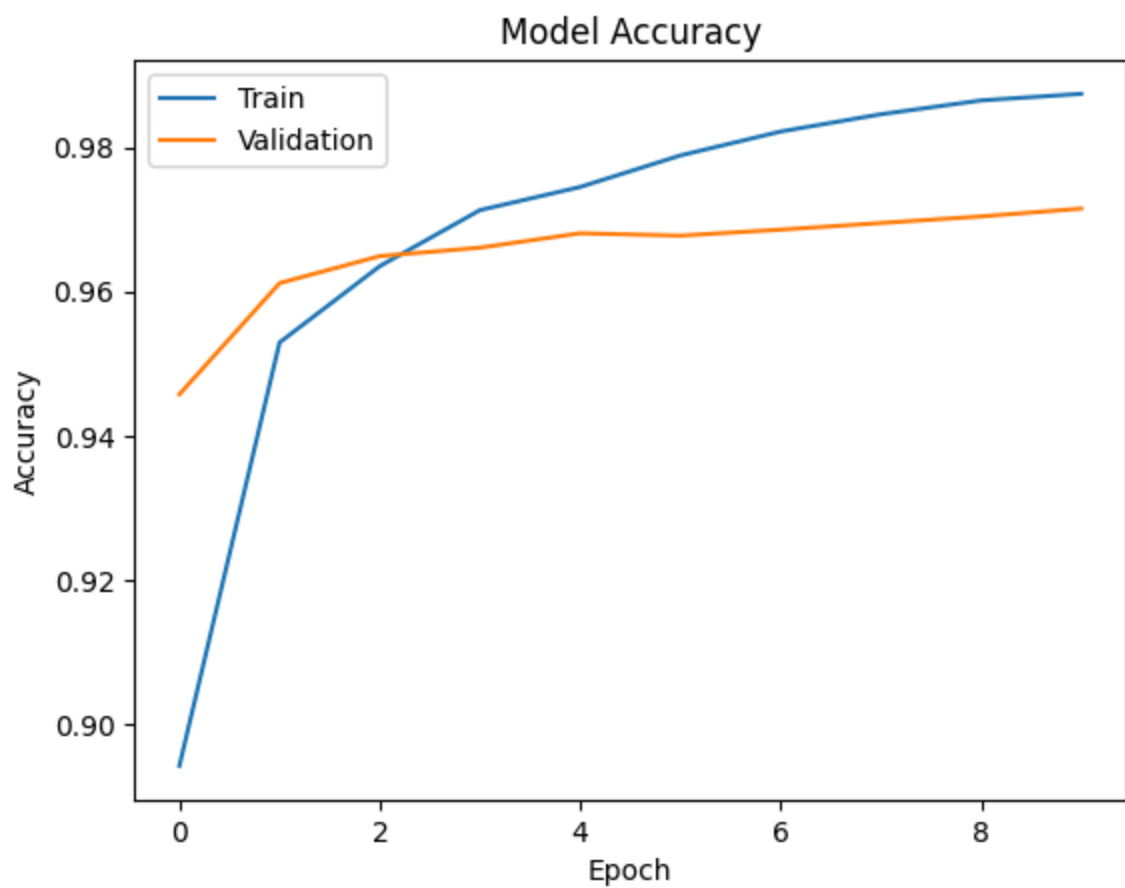
750/750 ————— **2s** 3ms/step - accuracy: 0.9867 - loss: 0.0433 - val_accuracy: 0.9704 - val_loss: 0.1079

Epoch 10/10

750/750 ————— **2s** 3ms/step - accuracy: 0.9881 - loss: 0.0372 - val_accuracy: 0.9715 - val_loss: 0.0997

313/313 ————— **1s** 2ms/step - accuracy: 0.9701 - loss: 0.0960

Test accuracy: 0.9743000268936157



```

In [ ]: import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense
        import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model with three hidden layers
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)), # Hidden Layer 1
    Dense(64, activation='relu'), # Hidden Layer 2
    Dense(32, activation='relu'), # Hidden Layer 3
    Dense(10, activation='softmax') # Output Layer
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train model
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# Accuracy Graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph

```

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
dense_19 (Dense)	(None, 128)	100,480
dense_20 (Dense)	(None, 64)	8,256
dense_21 (Dense)	(None, 32)	2,080
dense_22 (Dense)	(None, 10)	330

Total params: 111,146 (434.16 KB)

Trainable params: 111,146 (434.16 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

750/750  **4s** 4ms/step - accuracy: 0.8232 - loss: 0.5966 - val_accuracy: 0.9579 - val_loss: 0.1430

Epoch 2/10

750/750  **3s** 4ms/step - accuracy: 0.9606 - loss: 0.1321 - val_accuracy: 0.9663 - val_loss: 0.1162

Epoch 3/10

750/750  **3s** 4ms/step - accuracy: 0.9740 - loss: 0.0886 - val_accuracy: 0.9688 - val_loss: 0.1011

Epoch 4/10

750/750  **3s** 4ms/step - accuracy: 0.9805 - loss: 0.0637 - val_accuracy: 0.9736 - val_loss: 0.0899

Epoch 5/10

750/750  **3s** 4ms/step - accuracy: 0.9835 - loss: 0.0507 - val_accuracy: 0.9696 - val_loss: 0.1097

Epoch 6/10

750/750  **4s** 5ms/step - accuracy: 0.9880 - loss: 0.0381 - val_accuracy: 0.9731 - val_loss: 0.0980

Epoch 7/10

750/750  **3s** 4ms/step - accuracy: 0.9881 - loss: 0.0363 - val_accuracy: 0.9737 - val_loss: 0.1018

Epoch 8/10

750/750  **3s** 4ms/step - accuracy: 0.9906 - loss: 0.0282 - val_accuracy: 0.9729 - val_loss: 0.1134

Epoch 9/10

750/750  **3s** 4ms/step - accuracy: 0.9932 - loss: 0.0222 - val_accuracy: 0.9754 - val_loss: 0.0996

Epoch 10/10

750/750  **4s** 5ms/step - accuracy: 0.9949 - loss: 0.0163 - val_accuracy: 0.9727 - val_loss: 0.1175

313/313  **1s** 2ms/step - accuracy: 0.9698 - loss: 0.1164

Test accuracy: 0.9743000268936157

